In [24]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
import numpy as np
import matplotlib.pyplot as plt
```

In [25]:
```
data = load_iris()
X = data.data
y = data.target
print("Dataset Loaded Successfully")
```

```
Dataset Loaded Successfully
```

In [26]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Data Split Done")
```

```
Data Split Done
```

In [28]:
```
models = {
    "KNN": KNeighborsClassifier(),
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(),
    "SVM": SVC(),
    "Naive Bayes": GaussianNB()
}
print("Models Ready")
```

```
Models Ready
```

In [29]:
```
for name, model in models.items():
    print(f"\n ◆  {name} Results:")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred, average='macro'))
    print("Recall:", recall_score(y_test, y_pred, average='macro'))
    print("F1 Score:", f1_score(y_test, y_pred, average='macro'))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
🔹 KNN Results:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

🔹 Logistic Regression Results:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

🔹 Decision Tree Results:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

🔹 SVM Results:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

🔹 Naive Bayes Results:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

In [32]:

```
for name, model in models.items():
    print(f"\n📊 Classification Report for {name}:")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(classification_report(y_test, y_pred))
```

```
📊 Classification Report for KNN:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


📊 Classification Report for Logistic Regression:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


📊 Classification Report for Decision Tree:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


📊 Classification Report for SVM:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


📊 Classification Report for Naive Bayes:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
```

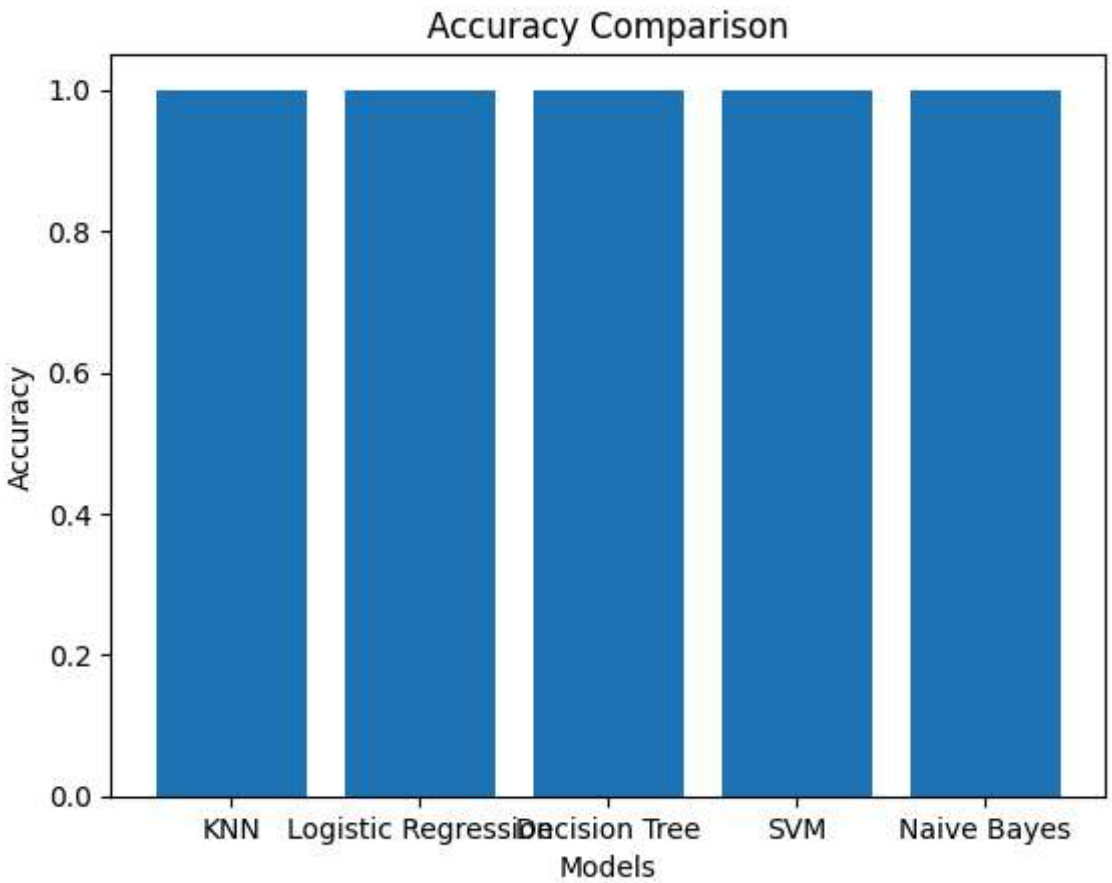
```
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

In [34]:
```
model_names = []
accuracies = []
precisions = []
recalls = []
f1_scores = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    model_names.append(name)
    accuracies.append(accuracy_score(y_test, y_pred))
    precisions.append(precision_score(y_test, y_pred, average='macro'))
    recalls.append(recall_score(y_test, y_pred, average='macro'))
    f1_scores.append(f1_score(y_test, y_pred, average='macro'))
model_names = ["KNN", "Logistic Regression", "Decision Tree", "SVM", "Naive Bayes"]
accuracies  = [1.0,   1.0,                   1.0,             1.0,   1.0]
```

In [35]:
```
plt.figure()
plt.bar(model_names, accuracies)
plt.title("Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.show()

# same repeated for Precision, Recall, F1 Score
```



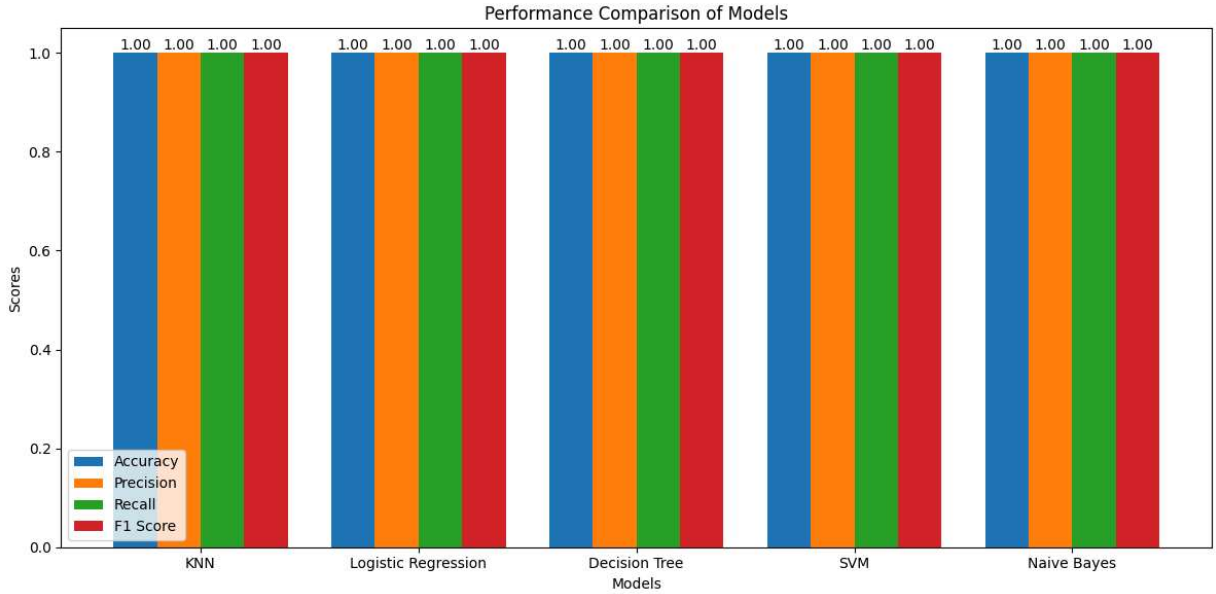
In [36]:
```
x = np.arange(len(model_names))
width = 0.2

plt.figure(figsize=(12, 6))
b1 = plt.bar(x - width,     accuracies,  width, label='Accuracy')
b2 = plt.bar(x,             precisions,  width, label='Precision')
b3 = plt.bar(x + width,     recalls,     width, label='Recall')
b4 = plt.bar(x + 2*width,   f1_scores,   width, label='F1 Score')

plt.xlabel("Models")
plt.ylabel("Scores")
plt.title("Performance Comparison of Models")
plt.xticks(x + width/2, model_names)
plt.legend()

def add_labels(bars):
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, height,
                 f'{height:.2f}', ha='center', va='bottom')

add_labels(b1)
add_labels(b2)
add_labels(b3)
add_labels(b4)
plt.tight_layout()
plt.show()
```


Performance Comparison of Models
1.00 1.00 1.00 1.00
1.00 1.00 1.00 1.00
1.00 1.00 1.00 1.00
1.00 1.00 1.00 1.00
1.00 1.00 1.00 1.00
Scores
1.0
0.8
0.6
0.4
0.2
0.0
Accuracy
Precision
Recall
F1 Score
KNN
Logistic Regression
Decision Tree
SVM
Naive Bayes
Models

In [ ]: